FUNAR
Technologien

Haskell

- statisches Typsystem, Typklassen
- effektvolle Berechnungen über explizite Monaden
- viele Erweiterungen
- nicht-strikte Auswertung
- viele Optionen für Parallelisierung und Nebenläufigkeit
- eigene Runtime
- hochoptimierender Compiler (Native Code)
- weltweite Community entwickelt Compiler

Beispiel

```haskell
data DecodeError
  = Field String DecodeError
  | Index Int DecodeError
  | OneOf [DecodeError]
  | Failure String Json.Value
  deriving (Show, Eq)

newtype Decoder a = Decoder {runDecoder :: Json.Value -> Either DecodeError a}

instance Functor Decoder where
  fmap f (Decoder decode) = Decoder (fmap (fmap f) decode)

instance Applicative Decoder where
  pure :: a -> Decoder a
  pure a = Decoder (pure (pure a))
  (<*>) :: Decoder (a -> b) -> Decoder a -> Decoder b
  Decoder decodeF <*> Decoder decodeA =
    Decoder ((<*>) <$> decodeF <*> decodeA)

instance Monad Decoder where
  (>>=) :: Decoder a -> (a -> Decoder b) -> Decoder b
  Decoder decode >>= f =
    Decoder ((>>=) <$> decode <*> flip (runDecoder . f))
```

- statisches Typsystem
- mächtiges Modulsystem
- imperative Features
- eigene Runtime
  (Bytecode, Native Code)
- optimierender Compiler
- weltweite Community,
  Entwicklung koordiniert von INRIA

Beispiel

https://github.com/janestreet/async_smtp

```
module Spoolable = struct
 module type S = sig
  (** [Spoolable.Metadata.t] should be smallish since it is read and written more
     frequently than [Spoolable.Data.t]. *)
  module Metadata : sig
   type t

   (** [of_string] and [to_string] are used to persist and read [t] on disk. *)
   include
    Stringable.S with type t := t
  end

  (** [Spoolable.Data.t] is where the "real" data lives and it allows for data-specific
     [load] and [save] functionality. *)
  module Data : sig
   type t

   val load : string -> t Deferred.Or_error.t
   val save : ?temp_file:string -> t -> string -> unit Deferred.Or_error.t
  end

  (** [Queue.t] is an enumerable type that represents the available queues and the
     mapping to directory names on-disk. *)
  module Queue : sig
   type t [@@deriving sexp, enumerate, compare]

   val to_dirname : t -> string
  end

  module Name_generator : Name_generator.S

  (** All operations that touch disk are passed through [Throttle.enqueue] *)
  module Throttle : sig
   val enqueue : (unit -> 'a Deferred.t) -> 'a Deferred.t
  end
```

@ctive group

Beispiel

```
module Make_base (S : Multispool_intf.Spoolable.S) = struct
 include Shared

 type t = spool [@@deriving sexp]

 let dir t = t

 let load_metadata path =
  S.Throttle.enqueue (fun () ->
   Deferred.Or_error.try_with (fun () ->
    let%bind contents = Reader.file_contents path in
    return (S.Metadata.of_string contents)))

 let save_metadata ?temp_file ~contents path =
  S.Throttle.enqueue (fun () ->
   Deferred.Or_error.try_with (fun () ->
    Writer.save path ?temp_file ~contents ~fsync:true))

 module Data_file = struct
  type t =
   { spool : spool
   ; name : string
   }

  let create spool name = { spool; name }
  let path t = data_dir_of t.spool ^/ t.name
  let load t = S.Throttle.enqueue (fun () -> S.Data.load (path t))
  let save t ~contents = S.Throttle.enqueue (fun () -> S.Data.save contents (path t))

  let stat t =
   Deferred.Or_error.try_with (fun () ->
    S.Throttle.enqueue (fun () -> Unix.stat (data_dir_of t.spool ^/ t.name)))
end
```

@ctive group

F#

- .NET-Abkömmling von OCaml
- allerdings ohne dessen Modulsystem
- Syntax für Applicatives / Monaden
- statisches Typsystem
- … mit Maßeinheiten
- integriert in Visual Studio
- *type providers*
- Entwicklung bei Microsoft koordiniert

Beispiel

http://fsharp.github.io/FSharp.Data/

```
#r "FSharp.Data.dll"
open FSharp.Data

let wb = WorldBankData.GetDataContext()
let uk = wb.Countries.``United Kingdom``
uk.Indicators.
```

- Average grace period on new external debt commitments, official (years)
- Average grace period on new external debt commitments, private (years)
- Average grant element on new external debt commitments (%)
- Average grant element on new external debt commitments, official (%)
- Average grant element on new external debt commitments, private (%)
- Average interest on new external debt commitments (%)
- Average interest on new external debt commitments, official (%)
- Average interest on new external debt commitments, private (%)
- Average maturity on new external debt commitments (years)

- dynamisch getypt
- Actor-Modell
- inhärent verteilt
- spezialisiert auf fehlertolerante Systeme
- umfangreiches Ökosystem
  für Deployment und Betrieb
- eigene Runtime
  (Bytecode, auch Native Code)
- Entwicklung koordiniert von Ericsson

Beispiel

https://github.com/zotonic/zotonic

```
-module(mod_tkvstore).
-author("Marc Worrell <marc@worrell.nl>").
-behaviour(gen_server).

-include_lib("zotonic_core/include/zotonic.hrl").


%% @doc Fetch the persistent data of a type/key
pid_observe_tkvstore_get(Pid, #tkvstore_get{} = Message, _Context) ->
   gen_server:call(Pid, Message).

%% @doc Fetch persistent data, first check the data dict that is still being written
handle_call(#tkvstore_get{type=Type, key=Key}, _From, State) ->
   case dict:find({Type, Key}, State#state.data) of
      {ok, Data} ->
         % Data is being written, return the data that is not yet in the store
         {reply, Data, State};
      error ->
         {reply, m_tkvstore:get(Type, Key, State#state.context), State}
   end;
```

elixir

- alternative Sprache für die Erlang-Plattform
- inspiriert durch Ruby
- interoperabel mit Erlang
- eigenes Build-/Package-Tooling
- Ruby-on-Rails-artiges Web-Framework

Beispiel

https://github.com/elixir-ecto/ecto

```
defmodule Ecto.Repo.Registry do

  use GenServer

  def associate(pid, value) when is_pid(pid) do
    GenServer.call(__MODULE__, {:associate, pid, value})
  end

  @impl true
  def handle_call({:associate, pid, value}, _from, table) do
    ref = Process.monitor(pid)
    true = :ets.insert(table, {pid, ref, value})
    {:reply, :ok, table}
  end

  @impl true
  def handle_info({:DOWN, ref, _type, pid, _reason}, table) do
    [{^pid, ^ref, _}] = :ets.lookup(table, pid)
    :ets.delete(table, pid)
    {:noreply, table}
  end
end
```

@ctive group

Swift

- hybride OO-/FP-Sprache
- statisches Typsystem
- entwickelt von Apple
- eng mit Objective-C / Cocoa integriert
- eigene Runtime / Reference Counting
- Native Code

Beispiel

```
indirect enum Diagram {
  case primitive(CGSize, Primitive)
  case beside(Diagram, Diagram)
  case below(Diagram, Diagram)
  case attributed(Attribute, Diagram)
  case align(CGPoint, Diagram)
}
extension Diagram {
  var size: CGSize {
    switch self {
      case .primitive(let size, _): return size
      case .attributed(_, let x): return x.size
      case let .beside(l, r):
        let sizeL = l.size
        let sizeR = r.size
        return CGSize(width: sizeL.width + sizeR.width,
          height: max(sizeL.height, sizeR.height))
      case let .below(l, r):
        return CGSize(width: max(l.size.width, r.size.width),
          height: l.size.height + r.size.height)
      case .align(_, let r): return r.size
    }
  }
}
```

Racket

- Baukasten für Sprachen
- enthält mehrere spezielle Lehrsprachen
- Abkömmling von Scheme/Lisp
- ungetypte und getypte Varianten
- eigene Runtime
  (Byte-Code, Native Code, JIT)

Beispiel

https://racket-lang.org/

#lang racket

```
(require 2htdp/image) ; draw a picture
(let sierpinski ([n 8])
  (cond
    [(zero? n) (triangle 2 'solid 'red)]
    [else (define t (sierpinski (- n 1)))
```

#lang scribble/base

```
@; Generate a PDF or HTML document
@title{Bottles: @italic{Abridged}}
@(apply
  itemlist
  (for/list ([n (in-range 100 0 -1)])
```

#lang typed/racket

```
;; Using higher-order occurrence typing
(define-type SrN (U String Number))
(: tog ((Listof SrN) -> String))
(define (tog l)
  (apply string-append (filter string? l)))
```

#lang datalog

```
ancestor(A, B) :- parent(A, B).
ancestor(A, B) :-
  parent(A, C), ancestor(C, B).
parent(john, douglas).
parent(bob, john).
```

#lang racket/gui

```
(define f (new frame% [label "Guess"]))
(define n (random 5)) (send f show #t)
(define ((check i) btn evt)
  (message-box "." (if (= i n) "Yes" "No")))
(for ([i (in-range 5)])
```

#lang web-server/insta

```
;; A "hello world" web server
(define (start request)
  (response/xexpr
   '(html
     (head (title "Racket"))
```

@ctive group

- hybride OO-/FP-Sprache
- statisches Typsystem
- Java-Plattform
- Entwicklung koordiniert bei
  Akka.io (Scala 2)
  EPFL (Scala 3)
- Fokus auf Weiterentwicklung

Beispiel

```
enum GameEvent {
        case HandDealt(player: Player, hand: Hand)
        case PlayerTurnChanged(player: Player)
        case LegalCardPlayed(player: Player, card: Card)
        case IllegalCardAttempted(player: Player, card: Card)
        case TrickTaken(player: Player, trick: Trick)
        case GameEnded(won: Player)
}

object GameEvent {
  def eventsWinner(events: Iterable[GameEvent]): Option[Player] =
                events.find({ case GameEnded(won) => true
                  case _ => false }) match {
                  case Some(GameEnded(won)) => Some(won)
                  case _ => None
                }
}
```

@ctive group

Kotlin

- primär OO-Sprache
- Funktionstypen
  (keine “Higher-Kinded Types”)
- statisches Typsystem
- entwickelt von JetBrains
- JVM

Beispiel

```
sealed interface Validation<out A> {
    fun <B> map(f: (A) -> B): Validation<B> =
        when (this) {
            is Valid -> Valid(f(this.value))
            is Invalid ->
                Invalid(this.errors)
        }

    fun <B> amap(vf: Validation<(A) -> B>): Validation<B> =
        when (this) {
            is Valid ->
                when (vf) {
                    is Valid ->
                        Valid(vf.value(this.value))
                    is Invalid -> Invalid(vf.errors)
                }
            is Invalid ->
                when (vf) {
                    is Valid -> Invalid(this.errors)
                    is Invalid -> Invalid(append(this.errors, vf.errors))
                }
        }
}
data class Valid<A>(val value: A): Validation<A>
data class Invalid(val errors: List<String>): Validation<Nothing>
```

@ctive group

Clojure

- modernes Lisp
- dynamisch getypt
- Java-Plattform
- Entwicklung koordiniert bei Cognitect
- Fokus auf Stabilität

Beispiel

https://github.com/active-group/vfei

```
; value is a map
(define-record-type ListFormat
  (^{:doc "Make a list format for a VFEI data item."}
   make-list-format size)
  list-format?
  [size list-format-size])

(define-record-type ArrayFormat
  (^{:doc "Make an array format for a VFEI data item."}
   make-array-format element-format size)
  array-format?
  [element-format array-format-element-format
   size array-format-size])

(define-record-type DataItem
  (^{:doc "Make a VFEI data item."}
   really-make-data-item name format value)
  data-item?
  [name data-item-name
   format data-item-format
   value data-item-value])

(defn make-data-item
  "Assert that list length matches format."
  [name format value]
  (cond
    (list-format? format)
    (when (not= (list-format-size format) (count value))
      (c/error `make-data-item "list length does not match number of parsed values" name format (count value) value))
    (array-format? format)
    (when (not= (array-format-size format) (count value))
      (c/error `make-data-item "array size does not match number of parsed values" name format (count value) value)))
  (really-make-data-item name format value))
```

@ctive group

Nach Plattform

Plattform	Sprache
Java	Clojure
Java	Scala
.NET	F#
Apple, Linux	Swift
Android	Kotlin
Erlang (Unix, Windows)	Elixir
Erlang	Erlang
Haskell (Unix, Windows)	native
OCaml (Unix)	native
Racket (Unix, Windows, Mac)	Racket

@ctive group

Statisch/dynamisch getypt

Typsystem	Sprache
dynamisch	Clojure
dynamisch	Elixir
dynamisch	Erlang
dynamisch (auch statisch)	Racket
statisch	F#
statisch, Typklassen	Haskell
statisch, Module	OCaml
statisch, OO-Integration	Scala
statisch, OO-Integration	Swift
statisch, OO-Integration	Kotlin

Grund pro Sprache

Grund	
Performance, große Projekte	OCaml
Performance, Parallelismus, fancy types	Haskell
Verteilung, Fehlertoleranz	Erlang, Elixir
.NET	F#
Java	Clojure, Scala
DSL	Racket
Apple	Swift
Android	Kotlin

Tail Calls vs. Plattform

Plattform	“Proper Tail Calls“
JavaScript	Standard: ✅, Implementierungen: 😢
Java (Scala, Clojure)	😢
.NET (F#)	✅
Racket, OCaml, Haskell, Erlang	✅
Swift	😢

Eingebettete DSLs

Sprache	DSL-Mechanismus
Racket	Makros
Clojure	Makros
Elixir	Makros
Haskell	Monaden + Typklassen
Scala	Monaden + Implicits
Kotlin	Monaden + Receiver Objects

Varianten (Auszug)

Sprache	Varianten
Haskell	Frege (JVM) ghc -> JS, Wasm
Haskell/Web	Purescript, Elm
OCaml	Reason (OCaml, JS), Js_of_ocaml (JS), wasm_of_ocaml
Scala	Scalajs (JS, Wasm)
Clojure	ClojureScript (JS)
F#	Fable (JS), Bolero (Wasm)
Kotlin	Kotlin for JavaScript, Kotlin/Wasm

@ctive group